GUIA COMPLETO DO ALUNO

Clínica veterinária | Banco e DER no MySQL Workbench | PHP + MySQL/MariaDB

Roteiro de preparação com criação visual do banco, código integral de todos os arquivos, explicação dos blocos, testes e organização das entregas. Mantém o projeto da clínica veterinária trabalhado nesta conversa; o caderno anexado sobre estoque corresponde a outro projeto.

Os menus descritos seguem o MySQL Workbench 8.x. Pode haver pequenas diferenças de posição entre versões. O Workbench é o cliente de modelagem e administração: não substitui o serviço MySQL/MariaDB nem o Apache.

Material de estudo para executar antes da prova. Durante a aplicação, respeite as orientações do avaliador, inclusive a regra de não utilizar internet. O Anexo 1 citado na prova não aparece no arquivo recebido; se for disponibilizado, use-o para apresentar a documentação.

Como utilizar este material

Siga as etapas na ordem. Ao terminar cada etapa, execute a conferência indicada antes de avançar. Crie os arquivos com os nomes exatos e salve em UTF-8 sem BOM. As linhas longas de código podem continuar visualmente na linha seguinte do PDF; o ZIP contém os mesmos arquivos para copiar sem quebras de apresentação.

Parte	O que será produzido
Planejamento	Requisitos, regras e organização das pastas
Banco e DER	Modelo .mwb, quatro tabelas, duas FKs e DER exportado
População e SQL	Três registros por tabela e saep_db.sql completo
Aplicação	12 arquivos PHP/CSS, criados e explicados um a um
Validação e entrega	Casos de teste, documentação e pasta compactada

1. Leia a situação e levante os requisitos

Problema: a clínica mantém cadastros e consultas manualmente, dificultando busca, organização dos horários e proteção dos dados. Objetivo: gerenciar tutores, pets e agendamentos com autenticação e controle de sessão.

RF01 - Autenticar por login e senha e informar tentativas inválidas.

RF02 - Mostrar nome do usuário autenticado e menu principal.

RF03 - Permitir sair e expirar sessão após inatividade.

RF04 - Listar, cadastrar, editar e excluir tutores.

RF05 - Pesquisar tutores por nome, e-mail ou telefone.

RF06 - Gerenciar pets com seleção obrigatória de tutor existente.

RF07 - Gerenciar agendamentos associados a um pet existente.

RF08 - Listar agendamentos por data, trazendo todos os dados do tutor e do pet.

RF09 - Validar os campos antes de gravar os cadastros.

Regras de negócio e decisões do exemplo

A prova exige pelo menos três registros em cada tabela dentro do script de criação e população.

CPF fica criptografado no banco e a senha usa hash. A senha não é armazenada como texto puro.

Cada pet pertence a um tutor. Cada agendamento pertence a um pet.

O exemplo bloqueia exclusão de tutor com pets e de pet com agendamentos, preservando os vínculos.

A expiração adotada é de 15 minutos sem requisições a páginas protegidas. A prova pede tempo de expiração, mas não determina esse número.

CPF de cadastro aceita 11 dígitos; o exemplo não valida os dígitos verificadores. Raça, nascimento, telefone e e-mail são obrigatórios nesta implementação. Observações da consulta podem ficar vazias.

2. Prepare os serviços e organize o trabalho

1. No XAMPP, abra o painel e inicie Apache e MySQL. No Laragon, inicie o servidor web e o serviço do banco. Confira as portas reais; evite executar duas instâncias na mesma porta.

2. Verifique que o MySQL Workbench está instalado e abre normalmente. Os alunos precisam ter o software preparado antes da aula/prova.

3. Use PHP 8.0 ou superior com mysqli, mysqlnd, openssl e mbstring. MySQL 8 ou MariaDB 10.4+ atende ao SQL fornecido. O Workbench é destinado ao MySQL: o MariaDB do XAMPP pode causar avisos ou falhas em recursos administrativos.

4. Caso a conexão/geração visual não funcione com MariaDB, use uma instância MySQL preparada pelo professor. Se o SQL Editor conectar e apenas o assistente/exportação falhar, a alternativa pelo SQL do próprio Workbench é explicada adiante.

5. Na área de trabalho, crie a pasta com o nome completo do aluno. Dentro, crie a pasta sistema. Em uma pasta separada de apoio, guarde modelo_clinica.mwb, estrutura_workbench.sql e 02_populacao.sql.

6. No XAMPP, crie a pasta de execução C:\xampp\htdocs\sistema. No Laragon, crie C:\laragon\www\sistema. Se a instalação estiver em outro diretório, use a raiz web correspondente.

7. Escolha a pasta de execução como local de edição dos arquivos PHP/CSS. Ao entregar, copie a versão final dela para a pasta sistema na área de trabalho. Isso evita editar uma cópia e testar outra.

Arquivo final	Local e finalidade
documentacao.pdf	Raiz da pasta do aluno; requisitos e descrição técnica
DER_clinica.pdf ou .png	Raiz da pasta do aluno; imagem/PDF do diagrama
saep_db.sql	Raiz da pasta do aluno; CREATE e INSERT completos

Arquivo final	Local e finalidade
sistema/	Código-fonte da aplicação; 12 arquivos
modelo_clinica.mwb	Apoio editável; pode acompanhar a entrega se permitido

Exemplo: Área de trabalho > NOME COMPLETO DO ALUNO > sistema. O banco é um serviço separado: não coloque sua pasta de dados do MySQL dentro do ZIP. Entregue o SQL exportado.

3. Crie a conexão local no Workbench

1. Abra o Workbench. Na tela inicial, localize MySQL Connections e clique no botão +.

2. Connection Name: Clínica SAEP. Connection Method: Standard (TCP/IP). Hostname: 127.0.0.1. Port: a porta do banco, normalmente 3306. Username: seu usuário, normalmente root no ambiente local.

3. Se existir senha, use Store in Vault ou informe quando solicitado. Não invente senha: confira a configuração da máquina.

4. Clique Test Connection. Se funcionar, confirme com OK. Se houver falha, confira serviço iniciado, porta, host e credenciais.

5. Dê dois cliques na conexão criada. Abra uma aba SQL e execute SELECT VERSION(); para identificar se o serviço é MySQL ou MariaDB.

6. Registre a versão real para a documentação. A versão do Workbench é diferente da versão do servidor.

```
SELECT VERSION();
SHOW DATABASES;
```

Conferência: a conexão deve permitir executar uma consulta. Ainda não existe saep_db se você estiver iniciando do zero.

4. Crie o modelo visual antes do banco

1. Escolha File > New Model. O modelo fica em uma aba própria, diferente da aba da conexão SQL.

2. No painel Physical Schemas, dê dois cliques no schema padrão, geralmente mydb. Altere o nome para saep_db.

3. Defina character set utf8mb4 e collation utf8mb4_unicode_ci. Essa collation evita depender de opções exclusivas do MySQL 8 ao usar MariaDB.

4. Salve pelo menu File > Save Model As, com o nome modelo_clinica.mwb na pasta de apoio.

5. Em EER Diagrams, dê dois cliques em Add Diagram. Será aberto um diagrama vazio.

6. Na barra vertical, escolha Place a New Table, clique na área do diagrama e dê dois cliques na tabela criada. O editor da tabela abre na parte inferior.

7. No campo Table Name, escreva o nome da tabela. Use Engine InnoDB. Na aba Columns, insira os campos indicados nas etapas seguintes.

Marca	Como usar neste projeto
PK	Primary Key: marque somente em id de cada tabela
NN	Not Null: campo obrigatório no banco
AI	Auto Increment: marque somente nos id das tabelas
UQ	Unique: use no login para não repetir nomes de acesso
UN	Unsigned: deixe desmarcado em todos os INT para manter os tipos iguais
FK	Foreign Key: configure na aba Foreign Keys; não é AI nem outra PK

Importante: desenhar a tabela e salvar .mwb não cria nada no servidor. A transferência do modelo para o banco ocorrerá em Forward Engineer. Ao criar tabelas no modelo, não procure o botão Apply do editor de tabelas do banco já conectado: são contextos diferentes.

5. Crie usuarios no modelo

1. Coloque uma tabela no diagrama e abra seu editor. Table Name: usuarios. Engine: InnoDB.

2. Na aba Columns, clique na primeira linha vazia e digite id. Datatype: INT. Marque PK, NN e AI. Deixe UQ e UN desmarcados para esse campo.

3. Na linha seguinte, digite nome; use VARCHAR(100) e marque NN.

4. Crie login com VARCHAR(60), NN e UQ. Crie senha_hash com VARCHAR(255) e NN.

5. Confira a aba Indexes: deve existir a PRIMARY em id e um índice UNIQUE em login. Se não aparecer UNIQUE, crie uq_usuarios_login, Type UNIQUE, contendo login.

6. Salve o modelo. Não escreva 123456 no campo senha_hash: ele receberá um hash pronto na população.

Coluna	Tipo	Marcações
id	INT	PK, NN, AI
nome	VARCHAR(100)	NN
login	VARCHAR(60)	NN, UQ
senha_hash	VARCHAR(255)	NN

6. Crie tutores no modelo

1. Escolha novamente Place a New Table e coloque a tabela ao lado de usuarios.

2. Nome: tutores. Engine: InnoDB.

3. Na aba Columns, cadastre os cinco campos conforme a tabela.

4. No id, marque PK, NN e AI. Nos outros quatro, marque NN.

5. cpf_criptografado usa TEXT porque guarda o resultado cifrado, com IV, tag e Base64; ele não contém apenas os 11 dígitos originais.

6. Salve e confirme que tutores possui somente uma PK: id.

Coluna	Tipo	Marcações
id	INT	PK, NN, AI
nome	VARCHAR(120)	NN
cpf_criptografado	TEXT	NN
telefone	VARCHAR(20)	NN
email	VARCHAR(120)	NN

7. Crie pets no modelo

1. Coloque a tabela abaixo de tutores. Nome: pets. Engine: InnoDB.

2. Insira os seis campos conforme a tabela.

3. Em id, marque PK, NN e AI. Em tutor_id, marque apenas NN: não marque PK nem AI.

4. Use INT em tutor_id e em tutores.id, mantendo UN desmarcado nos dois. Uma FK exige tipos compatíveis.

5. A tabela ainda precisa da FK. Criar a coluna com o nome tutor_id não estabelece uma relação sozinho.

Coluna	Tipo	Marcações
id	INT	PK, NN, AI
nome	VARCHAR(100)	NN
especie	VARCHAR(50)	NN
raca	VARCHAR(80)	NN
data_nascimento	DATE	NN
tutor_id	INT	NN; será FK

8. Configure a FK de pets para tutores

1. Dê dois cliques em pets para abrir o editor. Selecione a aba Foreign Keys.

2. Na lista Foreign Key Name, clique na última linha vazia e escreva fk_pet_tutor.

3. Em Referenced Table, selecione tutores do schema saep_db.

4. Na lista de colunas, marque tutor_id. Na coluna correspondente de referência, escolha id. O mapeamento deve ser pets.tutor_id → tutores.id.

5. Em Foreign Key Options, escolha ON DELETE RESTRICT e ON UPDATE CASCADE.

6. Salve e volte ao diagrama. Deve aparecer uma linha entre as duas tabelas.

7. Confira: um tutor pode ter zero ou vários pets; cada pet precisa ter exatamente um tutor. A FK não integra a PK do pet, por isso o relacionamento é não identificador.

Use a aba Foreign Keys para aproveitar tutor_id já existente. Alguns ícones de relacionamento criam automaticamente outra coluna, como tutores_id; isso deixa nomes diferentes do PHP deste roteiro. Se acontecer, corrija o modelo antes de gerar o banco, mantendo somente tutor_id e a FK correta.

9. Crie agendamentos e sua FK

Coluna	Tipo	Marcações
id	INT	PK, NN, AI
pet_id	INT	NN; será FK
data_hora	DATETIME	NN
motivo	VARCHAR(255)	NN
observacoes	TEXT	NN; texto vazio é aceito

1. Adicione a tabela agendamentos abaixo de pets. Use Engine InnoDB e os cinco campos da tabela.

2. Marque PK, NN e AI no id. Em pet_id, use INT e NN, sem PK, AI ou UN.

3. Abra Foreign Keys. Adicione fk_agendamento_pet e selecione Referenced Table pets.

4. Marque pet_id e mapeie para pets.id. Configure DELETE RESTRICT e UPDATE CASCADE.

5. Salve e confirme a segunda linha do diagrama. Cada consulta tem um pet; um pet pode ter zero ou várias consultas.

6. Deixe usuarios separada. Não é necessária uma FK dela para tutor, pet ou agendamento neste modelo.

Por que não criar tutor_id em agendamentos? O tutor é obtido pelo pet: agendamentos.pet_id identifica pets.id e pets.tutor_id identifica tutores.id. O JOIN reúne os dados sem repetir esse vínculo na tabela de consultas.

10. Revise o DER e exporte sua imagem

1. Arraste tutores, pets e agendamentos para uma sequência vertical e mantenha usuarios ao lado. Não sobreponha tabelas.

2. Confira nomes, tipos, PKs, NN, AI e as duas linhas de relação. As FKs precisam apontar para id das tabelas pai.

3. Salve novamente modelo_clinica.mwb.

4. Com o EER Diagram ativo, abra File > Export e escolha exportação em PNG ou PDF disponível na sua versão. Salve DER_clinica.png ou DER_clinica.pdf na pasta final do aluno.

5. Abra o arquivo exportado fora do Workbench. Veja se todos os campos e as duas relações estão legíveis e sem cortes.

6. O pacote inclui um DER de referência. O aluno deve praticar a exportação do modelo que acabou de construir.

11. Gere o banco pelo Forward Engineer

1. Com o modelo ativo, escolha Database > Forward Engineer. Esse procedimento envia o modelo para uma conexão real.

2. Escolha a conexão Clínica SAEP criada anteriormente. Confira host, porta e usuário e avance.

3. Na tela de opções, mantenha a criação do schema. Não escolha opções de DROP para apagar um banco já existente.

4. Se houver a opção Skip Creation of FOREIGN KEYS, deixe-a desmarcada. As FKs precisam ser geradas.

5. Selecione as quatro tabelas. Avance para a revisão do SQL antes de executar.

6. Confira que o script utiliza saep_db, InnoDB, nomes corretos das colunas, UNIQUE no login e duas FOREIGN KEY.

7. Execute o assistente e confira o relatório final. Um erro não deve ser ignorado: volte ao modelo e corrija a causa.

8. Retorne à aba de conexão SQL, atualize SCHEMAS e expanda saep_db > Tables. Devem aparecer usuarios, tutores, pets e agendamentos.

Execute estas consultas na conexão SQL. Selecione o trecho inteiro e use o raio para executar a seleção, ou Ctrl+Shift+Enter. Para apenas a instrução sob o cursor, use Ctrl+Enter. Consulte o painel Action Output e o Result Grid.

```
USE saep_db;
SHOW TABLES;
SHOW CREATE TABLE pets;
SHOW CREATE TABLE agendamentos;
```

Conferência: o resultado precisa mostrar fk_pet_tutor e fk_agendamento_pet. As tabelas estão vazias nesta etapa. Não execute saep_db.sql completo por cima dessas tabelas, pois ele também contém CREATE TABLE.

12. Alternativa: gere o SQL do modelo e execute no Workbench

Use esta alternativa se quiser revisar o arquivo antes de executar ou se o assistente direto apresentar falha. É um caminho alternativo à etapa 11; não execute os dois em sequência sobre as mesmas tabelas.

1. No modelo, abra File > Export > Forward Engineer SQL CREATE Script.

2. Escolha estrutura_workbench.sql na pasta de apoio. Inclua criação do schema e das quatro tabelas; não omita FKs.

3. Não inclua DROP para dados que você precisa preservar. Revise o script, especialmente nomes e collation.

4. Salve o arquivo. Na conexão SQL, escolha File > Open SQL Script e abra estrutura_workbench.sql.

5. Execute o script completo em um schema ainda não criado ou sem as tabelas. Atualize SCHEMAS e confira as tabelas.

6. Se a geração inserir um nome de collation que o MariaDB não reconhece, configure utf8mb4_unicode_ci no modelo e gere novamente.

7. O arquivo 01_estrutura_referencia.sql no pacote mostra o DDL equivalente para comparação. Ele não substitui a prática da criação visual do modelo.

DDL esperado para conferência

```
CREATE DATABASE IF NOT EXISTS saep_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE saep_db;
CREATE TABLE usuarios (
 id INT AUTO_INCREMENT PRIMARY KEY,
 nome VARCHAR(100) NOT NULL,
 login VARCHAR(60) NOT NULL UNIQUE,
 senha_hash VARCHAR(255) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE tutores (
 id INT AUTO_INCREMENT PRIMARY KEY,
 nome VARCHAR(120) NOT NULL,
 cpf_criptografado TEXT NOT NULL,
 telefone VARCHAR(20) NOT NULL,
 email VARCHAR(120) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE pets (
 id INT AUTO_INCREMENT PRIMARY KEY,
 nome VARCHAR(100) NOT NULL,
 especie VARCHAR(50) NOT NULL,
 raca VARCHAR(80) NOT NULL,
 data_nascimento DATE NOT NULL,
 tutor_id INT NOT NULL,
 CONSTRAINT fk_pet_tutor FOREIGN KEY (tutor_id)
 REFERENCES tutores(id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
CREATE TABLE agendamentos (
 id INT AUTO_INCREMENT PRIMARY KEY,
 pet_id INT NOT NULL,
 data_hora DATETIME NOT NULL,
 motivo VARCHAR(255) NOT NULL,
 observacoes TEXT NOT NULL,
 CONSTRAINT fk_agendamento_pet FOREIGN KEY (pet_id)
 REFERENCES pets(id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

13. Insira os três registros de cada tabela

1. Na conexão SQL do Workbench, abra uma nova aba ou File > Open SQL Script > 02_populacao.sql.

2. Execute somente a população abaixo, depois de criar as tabelas. A primeira linha seleciona saep_db.

3. Observe a ordem: usuarios e tutores primeiro, depois pets, por último agendamentos. Os registros pai precisam existir antes das FKs.

4. Os INSERTs fornecidos usam exatamente a ordem de colunas das tabelas deste roteiro. Se você alterou a ordem, volte ao modelo para manter a correspondência ou use listas explícitas de colunas.

5. Os três usuários possuem a senha didática 123456. Os logins são admin, recepcao e vet. Os CPFs são fictícios, já cifrados para a chave do config.php.

6. Execute uma vez. Repetir os INSERTs com os mesmos IDs causará Duplicate entry. Para testar novamente, consulte os registros existentes em vez de inserir em duplicidade.

```
USE saep_db;
INSERT INTO usuarios VALUES (1, 'Administrador', 'admin',
'$2b$12$G3u2JYle3DwdtBB3JUfD4eMvYHkL2p2ry8D0zxTl0m2zj1KBWT0si');
INSERT INTO usuarios VALUES (2, 'Recepção', 'recepcao',
'$2b$12$IniF9Wusd.PODBe4pXHwxeoKz.i/ILI.uql7zmxgiBhVm9ZSaE3zy');
INSERT INTO usuarios VALUES (3, 'Veterinário', 'vet',
'$2b$12$XdUNZqjiTCxpedU45wvzZu6S7CyG2ebeNvpsEVp9la03SgkJUJbCW');
INSERT INTO tutores VALUES (1, 'Tutor Exemplo A',
'AmWNjmPjOb+GX/S6VH4ba2JYa4RhklQe9CtMLECO43puxoBYTf2E', '17999990001', 'tutor1@exemplo.com');
INSERT INTO tutores VALUES (2, 'Tutor Exemplo B',
'FpBdcWvQ3Uv18YmhV5/8ZzD/JLddWXjkvtRED8/hvmPAbJdSIY7j', '17999990002', 'tutor2@exemplo.com');
INSERT INTO tutores VALUES (3, 'Tutor Exemplo C',
'Z54/l7XrzL71VrjGwP0V3Xgq6Du1lRaxhyBdo6E4ioukq7SwToNx', '17999990003', 'tutor3@exemplo.com');
INSERT INTO pets VALUES
(1,'Thor','Cachorro','SRD','2022-01-10',1),
(2,'Luna','Gato','SRD','2023-03-15',2),
(3,'Mel','Cachorro','Poodle','2021-06-20',3);
INSERT INTO agendamentos VALUES
(1,1,'2026-10-10 08:00:00','Consulta','Primeiro atendimento'),
(2,2,'2026-10-10 09:00:00','Vacinação','Verificar carteira'),
(3,3,'2026-10-11 10:00:00','Retorno','Reavaliação');
```

Entenda o que foi gravado

senha_hash foi gerada com bcrypt; no PHP, password_verify verifica a senha digitada. O CPF foi cifrado com AES-256-GCM. O valor salvo corresponde a Base64 de IV + tag + texto cifrado. Base64 sozinho não protege um CPF. O segredo didático do config.php precisa ser o mesmo para recuperar esses exemplos.

Não substitua senha_hash por 123456 nem cpf_criptografado por números legíveis. Para cadastrar outros tutores, use o formulário, que chama criptografar(). Para gerar outro hash em PHP, o exemplo é password_hash("outra_senha", PASSWORD_DEFAULT).

Conferência da população

```
SELECT 'usuarios' AS tabela, COUNT(*) AS total FROM usuarios
UNION ALL SELECT 'tutores', COUNT(*) FROM tutores
UNION ALL SELECT 'pets', COUNT(*) FROM pets
UNION ALL SELECT 'agendamentos', COUNT(*) FROM agendamentos;
SELECT id, nome, login, senha_hash FROM usuarios;
SELECT id, nome, cpf_criptografado FROM tutores;
```

Esperado: quatro linhas com total 3. As senhas devem aparecer como hash e os CPFs como texto cifrado. Depois dos testes, os totais podem aumentar ou diminuir; mantenha no mínimo três registros por tabela no SQL final.

14. Exporte estrutura e dados para saep_db.sql

1. Na aba da conexão SQL, escolha Server > Data Export ou Data Export no Navigator.

2. Marque saep_db e as quatro tabelas. Se não aparecerem, use Refresh.

3. Escolha Dump Structure and Data. Não use Dump Structure Only: a prova exige também os registros.

4. Escolha Export to Self-Contained File e indique saep_db.sql dentro da pasta final do aluno.

5. Marque Include Create Schema se essa opção aparecer, para incluir a criação do banco.

6. Clique Start Export e confira Export Progress. Só considere terminado se não houver erro.

7. Abra saep_db.sql no editor. Procure CREATE DATABASE/CREATE SCHEMA ou garanta uma instrução equivalente para criar saep_db. Confira USE saep_db, quatro CREATE TABLE, duas FKs e INSERTs para as quatro tabelas.

8. Confirme que o arquivo exportado não contém CPFs ou senhas originais em texto puro. Exportar uma tabela para CSV não atende à entrega do script SQL completo.

Se o Data Export falhar, especialmente com MariaDB

1. O exportador do Workbench usa utilitários do MySQL; uma conexão SQL funcionando não garante que o dump funcione com MariaDB.

2. Use a estrutura gerada pelo modelo na etapa 12. Salve uma cópia como saep_db.sql na pasta final.

3. Abra esse saep_db.sql no editor e acrescente, no final, USE saep_db seguido de todo o conteúdo de 02_populacao.sql.

4. Isso forma um script de criação e população com os três registros fictícios por tabela. Não altera nem apaga o banco que já está em uso.

5. Se precisar entregar alterações realizadas nos testes, gere INSERTs correspondentes aos dados finais protegidos. Não acrescente novamente a população de exemplo se já existir no script.

6. O pacote contém saep_db.sql de referência, com estrutura e dados juntos. Use-o para conferir o resultado, mantendo o modelo e o código consistentes.

Uma checagem de restauração pode ser feita em uma instância de teste vazia. Não apague o banco da aula para testar. O arquivo usa saep_db; restaure em ambiente isolado ou trabalhe em uma cópia do script com outro schema e confira todas as referências de schema.

15. Prepare os arquivos da aplicação no editor

1. Abra VS Code ou outro editor. Use File > Open Folder para abrir a pasta sistema dentro de htdocs ou www.

2. Crie cada arquivo da lista abaixo. Use o nome exato, incluindo extensão. Não crie login.php.txt.

3. Salve cada arquivo em UTF-8 sem BOM. Em arquivos PHP, <?php deve ser o primeiro conteúdo.

4. Crie os arquivos auxiliares antes de testar os cadastros: eles dependem uns dos outros. Não abra o PHP com duplo clique ou endereço file://; use http://localhost/sistema.

5. Após concluir o código, inicie os serviços e teste na pasta de execução. A pasta do aluno na área de trabalho será atualizada apenas com a versão final.

Ordem	Arquivo	Função
1	config.php	Configuração do banco e sessão
2	conexao.php	Conexão mysqli
3	funcoes.php	Segurança e HTML compartilhado
4	index.php	Entrada da aplicação
5	proteger.php	Proteção das páginas privadas
6	login.php	Autenticação
7	principal.php	Menu e nome do usuário
8	logout.php	Encerramento de sessão
9	tutores.php	CRUD e busca de tutores
10	pets.php	CRUD de pets com tutor
11	agendamentos.php	CRUD de consultas e JOIN
12	style.css	Visual das páginas

16. Crie config.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva config.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a config.php.

```
<?php
// Ajuste estes dados conforme seu XAMPP ou Laragon.
const DB_HOST = '127.0.0.1';
const DB_PORT = 3306;
const DB_USER = 'root';
const DB_PASS = '';
const DB_NAME = 'saep_db';
const TEMPO_SESSAO = 900; // 15 minutos sem atividade.
// Chave didática, compatível com os CPFs de exemplo do SQL.
// Em produção, use segredo externo e planeje a migração dos dados.
const SEGREDO_CPF = 'SAEP-clinica-chave-didatica-2026';
date_default_timezone_set('America/Sao_Paulo');
```

Como o arquivo funciona

O que configurar

DB_HOST, DB_PORT, DB_USER, DB_PASS e DB_NAME devem corresponder ao servidor usado no Workbench. O PHP abre sua própria conexão: a conexão salva no Workbench não configura o PHP automaticamente.

Tempo e chave

TEMPO_SESSAO usa segundos. 900 = 15 minutos. SEGREDO_CPF é uma chave didática compatível com os INSERTs. Não mude depois de cifrar os dados sem planejar sua migração.

Conferência

Deixe DB_NAME como saep_db. A senha root vazia é só um exemplo comum de ambiente local; use a senha real da máquina.

17. Crie conexao.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva conexao.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a conexao.php.

```
<?php
require_once 'config.php';
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
try {
    $conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME, DB_PORT);
    $conn->set_charset('utf8mb4');
} catch (mysqli_sql_exception $e) {
    error_log($e->getMessage());
    exit('Falha ao conectar. Confira config.php e o serviço MySQL.');
}
```

Como o arquivo funciona

Importação

require_once carrega config.php uma única vez. new mysqli recebe os parâmetros e abre a conexão.

Erros e caracteres

mysqli_report permite capturar erros como exceções. set_charset utf8mb4 suporta acentos. O catch envia os detalhes ao log e apresenta uma mensagem curta.

Conferência

Se o login mostrar falha de conexão, resolva os dados de acesso e o serviço antes de modificar os formulários.

18. Crie funcoes.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva funcoes.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a funcoes.php.

```
<?php
require_once 'config.php';
if (session_status() !== PHP_SESSION_ACTIVE) {
    session_set_cookie_params([
        'httponly' => true, 'samesite' => 'Lax',
        'secure' => !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off'
    ]);
    session_start();
}
function e($valor) {
    return htmlspecialchars((string)$valor, ENT_QUOTES, 'UTF-8');
}
function token() {
    if (empty($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf'];
}
function validarToken() {
    if (!hash_equals(token(), $_POST['csrf'] ?? '')) {
        http_response_code(403);
        exit('Formulário inválido. Recarregue a página.');
    }
}
function criptografar($cpf) {
    $iv = random_bytes(12);
    $chave = hash('sha256', SEGREDO_CPF, true);
    $texto = openssl_encrypt($cpf, 'aes-256-gcm', $chave,
        OPENSSL_RAW_DATA, $iv, $tag);
    if ($texto === false) { throw new RuntimeException('Falha na criptografia.'); }
    return base64_encode($iv . $tag . $texto);
}
function descriptografar($valor) {
    $dados = base64_decode($valor, true);
    if ($dados === false || strlen($dados) < 28) { return ''; }
    $texto = openssl_decrypt(substr($dados, 28), 'aes-256-gcm',
        hash('sha256', SEGREDO_CPF, true), OPENSSL_RAW_DATA,
        substr($dados, 0, 12), substr($dados, 12, 16));
    return $texto === false ? '' : $texto;
}
function dataValida($valor, $formato) {
    $data = DateTime::createFromFormat('!' . $formato, $valor);
    return $data && $data->format($formato) === $valor;
}
function cabecalho($titulo) {
    echo '<!DOCTYPE html><html lang="pt-br"><head><meta charset="UTF-8">';
    echo '<meta name="viewport" content="width=device-width, initial-scale=1">';
    echo '<title>' . e($titulo) . '</title><link rel="stylesheet" href="style.css">';
    echo '</head><body><main><h1>' . e($titulo) . '</h1>';
    if (!empty($_SESSION['usuario_id'])) {
        echo '<nav><a href="principal.php">Principal</a> ';
        echo '<a href="tutores.php">Tutores</a> <a href="pets.php">Pets</a> ';
        echo '<a href="agendamentos.php">Agendamentos</a></nav>';
    }
}
function rodape() { echo '</main></body></html>'; }
```

Como o arquivo funciona

Sessão

session_start cria ou retoma a sessão antes de sair qualquer HTML. Os parâmetros do cookie são configurados antes da sessão.

Saída segura

e() converte caracteres especiais ao apresentar um valor em HTML. Isso não modifica o valor do banco.

Proteção do formulário

token() cria um código CSRF aleatório e guarda na sessão. validarToken() compara o enviado no POST; se não corresponder, retorna 403.

Criptografia

criptografar() gera um IV novo para cada CPF e usa AES-256-GCM com a chave derivada. descriptografar() extrai IV e tag e tenta recuperar o texto. Falhas retornam string vazia.

Datas e páginas

dataValida() confere o formato e datas reais. cabecalho() cria HTML, título e menu. rodape() fecha main, body e html.

Conferência

CPF da base deve aparecer corretamente após autenticar. O segredo de estudo deve coincidir com a população do SQL.

19. Crie index.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva index.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a index.php.

```
<?php
header('Location: login.php');
exit;
```

Como o arquivo funciona

Entrada

header envia o navegador para login.php. exit interrompe o arquivo para não gerar saída depois do redirecionamento.

Conferência

Abrir http://localhost/sistema deve mostrar o formulário de acesso.

20. Crie proteger.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva proteger.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a proteger.php.

```
<?php
require_once 'funcoes.php';
if (empty($_SESSION['usuario_id'])) {
    header('Location: login.php'); exit;
}
if (time() - ($_SESSION['ultima_atividade'] ?? 0) >= TEMPO_SESSAO) {
    $_SESSION = [];
    session_destroy();
    header('Location: login.php?expirou=1'); exit;
}
$_SESSION['ultima_atividade'] = time();
```

Como o arquivo funciona

Autorização

O arquivo inclui funcoes.php e exige usuario_id na sessão. Se não existir, volta ao login e encerra a execução.

Expiração

Compara time() com ultima_atividade. Ao ultrapassar o limite, limpa e destrói a sessão. Se válida, atualiza o horário.

Uso

Inclua antes de HTML em principal.php e nos três cadastros. A expiração é percebida na próxima requisição; não existe temporizador visível na tela.

21. Crie login.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva login.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a login.php.

```
<?php
require_once 'conexao.php';
require_once 'funcoes.php';
$erro = isset($_GET['expirou']) ? 'Sessão expirada. Entre novamente.' : '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    validarToken();
    $login = trim($_POST['login'] ?? '');
    $senha = $_POST['senha'] ?? '';
    $stmt = $conn->prepare('SELECT * FROM usuarios WHERE login = ?');
    $stmt->bind_param('s', $login);
    $stmt->execute();
    $usuario = $stmt->get_result()->fetch_assoc();
    if ($usuario && password_verify($senha, $usuario['senha_hash'])) {
        session_regenerate_id(true);
        $_SESSION['usuario_id'] = $usuario['id'];
        $_SESSION['usuario_nome'] = $usuario['nome'];
        $_SESSION['ultima_atividade'] = time();
        header('Location: principal.php'); exit;
    }
    $erro = 'Login ou senha incorretos.';
}
cabecalho('Clínica veterinária - Login');
?>
<p class="aviso"><?= e($erro) ?></p>
<form method="post">
 <input type="hidden" name="csrf" value="<?= e(token()) ?>">
 <label>Login <input name="login" required maxlength="60"></label>
 <label>Senha <input type="password" name="senha" required></label>
 <button>Entrar</button>
</form>
<?php rodape(); ?>
```

Como o arquivo funciona

Recepção dos dados

GET mostra a página. POST verifica token, recebe login com trim e mantém a senha exatamente como digitada.

Consulta preparada

SELECT procura somente pelo login. bind_param("s", $login) informa string. A senha não é concatenada nem comparada diretamente no SQL.

Validação

fetch_assoc traz a linha do usuário ou null. password_verify compara a senha com o hash. A condição exige os dois para autenticar.

Sucesso

session_regenerate_id renova o identificador. O código guarda ID, nome e horário; header envia à Principal e exit encerra o processamento.

Falha

A mensagem não revela se o erro era no login ou na senha. O formulário envia pelo método POST e inclui o token oculto.

Teste

Use admin / 123456 e depois uma senha errada. No primeiro caso abra Principal; no segundo mantenha Login com aviso.

22. Crie principal.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva principal.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a principal.php.

```
<?php
require_once 'proteger.php';
cabecalho('Principal');
?>
<p>Bem-vindo, <?= e($_SESSION['usuario_nome']) ?>!</p>
<p>Use o menu para cadastrar tutores, pets e agendamentos.</p>
<form action="logout.php" method="post">
 <input type="hidden" name="csrf" value="<?= e(token()) ?>">
 <button>Sair do sistema</button>
</form>
<?php rodape(); ?>
```

Como o arquivo funciona

Página privada

proteger.php verifica a sessão antes de montar a página. O nome vem de usuario_nome.

Navegação

cabecalho mostra os três links. O formulário de saída envia POST com token para logout.php.

Teste

Veja Administrador após acessar com admin. Abra Tutores, Pets e Agendamentos pelos links e retorne à Principal.

23. Crie logout.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva logout.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a logout.php.

```
<?php
require_once 'funcoes.php';
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: principal.php'); exit;
}
validarToken();
$_SESSION = [];
if (ini_get('session.use_cookies')) {
    $p = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $p['path'], $p['domain'], $p['secure'], $p['httponly']);
}
session_destroy();
header('Location: login.php');
exit;
```

Como o arquivo funciona

Método e token

A saída só ocorre por POST com token válido. Um GET redireciona à Principal.

Limpeza

Apaga dados da sessão, expira o cookie se utilizado e chama session_destroy. Depois retorna ao Login.

Teste

Clique Sair e tente abrir tutores.php pelo endereço. A página deve exigir novo login.

24. Crie tutores.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva tutores.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a tutores.php.

```
<?php
require_once 'proteger.php';
require_once 'conexao.php';
$erro = '';
$id = 0;
$nome = '';
$cpf = '';
$telefone = '';
$email = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    validarToken();
    $id = filter_var($_POST['id'] ?? 0, FILTER_VALIDATE_INT);
    try {
        if ($id === false || $id < 0) { throw new RuntimeException('ID inválido.'); }
        if (($_POST['acao'] ?? '') === 'excluir') {
            $stmt = $conn->prepare('DELETE FROM tutores WHERE id = ?');
            $stmt->bind_param('i', $id);
            $stmt->execute();
            header('Location: tutores.php?ok=1'); exit;
        }
        $nome = trim($_POST['nome'] ?? '');
        if ($nome === '') { throw new RuntimeException('Preencha Nome.'); }
        if (mb_strlen($nome) > 120) { throw new RuntimeException('Campo Nome muito longo.'); }
        $cpf = trim($_POST['cpf'] ?? '');
        if ($cpf === '') { throw new RuntimeException('Preencha CPF (11 dígitos).'); }
        if (mb_strlen($cpf) > 11) { throw new RuntimeException('Campo CPF (11 dígitos) muito longo.');
}
        $telefone = trim($_POST['telefone'] ?? '');
        if ($telefone === '') { throw new RuntimeException('Preencha Telefone.'); }
        if (mb_strlen($telefone) > 20) { throw new RuntimeException('Campo Telefone muito longo.'); }
        $email = trim($_POST['email'] ?? '');
        if ($email === '') { throw new RuntimeException('Preencha E-mail.'); }
        if (mb_strlen($email) > 120) { throw new RuntimeException('Campo E-mail muito longo.'); }
        if (!preg_match('/^[0-9]{11}$/', $cpf)) {
            throw new RuntimeException('CPF deve conter 11 dígitos numéricos.');
        }
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new RuntimeException('E-mail inválido.');
        }
        $cpf_criptografado = criptografar($cpf);
        if ($id > 0) {
            $stmt = $conn->prepare('UPDATE tutores SET nome = ?, cpf_criptografado = ?, telefone = ?,
email = ? WHERE id = ?');
            $stmt->bind_param('ssssi', $nome, $cpf_criptografado, $telefone, $email, $id);
        } else {
            $stmt = $conn->prepare('INSERT INTO tutores (nome, cpf_criptografado, telefone, email)
VALUES (?, ?, ?, ?)');
            $stmt->bind_param('ssss', $nome, $cpf_criptografado, $telefone, $email);
        }
        $stmt->execute();
        header('Location: tutores.php?ok=1'); exit;
    } catch (mysqli_sql_exception $ex) {
        error_log($ex->getMessage());
        $erro = 'Operação não concluída. Verifique os vínculos: registros com dependências não podem
ser excluídos.';
    } catch (RuntimeException $ex) {
        $erro = $ex->getMessage();
    }
}
if (isset($_GET['editar']) && $_SERVER['REQUEST_METHOD'] !== 'POST') {
    $id = intval($_GET['editar']);
    $stmt = $conn->prepare('SELECT * FROM tutores WHERE id = ?');
    $stmt->bind_param('i', $id);
    $stmt->execute();
```

```
    $registro = $stmt->get_result()->fetch_assoc();
    if (!$registro) { $id = 0; $erro = 'Registro não encontrado.'; }
    else {
        $nome = $registro['nome'];
        $cpf = descriptografar($registro['cpf_criptografado']);
        $telefone = $registro['telefone'];
        $email = $registro['email'];
    }
}
$busca = trim($_GET['busca'] ?? '');
$termo = '%' . $busca . '%';
$stmt = $conn->prepare('SELECT * FROM tutores WHERE nome LIKE ? OR email LIKE ? OR telefone LIKE ?
ORDER BY nome');
$stmt->bind_param('sss', $termo, $termo, $termo);
$stmt->execute();
$listagem = $stmt->get_result();
cabecalho('Tutores');
?>
<p class="aviso"><?= e($erro) ?></p>
<?php if (isset($_GET['ok'])): ?><p>Operação concluída.</p><?php endif; ?>
<form method="get"><label>Buscar por nome, e-mail ou telefone<input name="busca" value="<?= e($busca)
?>"></label><button>Buscar</button><a href="tutores.php">Limpar</a></form>
<h2><?= $id ? 'Editar registro' : 'Novo registro' ?></h2>
<form method="post">
<input type="hidden" name="csrf" value="<?= e(token()) ?>">
<input type="hidden" name="id" value="<?= e($id) ?>">
<label>Nome<input type="text" name="nome" value="<?= e($nome) ?>" maxlength="120" required></label>
<label>CPF (11 dígitos)<input type="text" name="cpf" value="<?= e($cpf) ?>" maxlength="11"
required></label>
<label>Telefone<input type="text" name="telefone" value="<?= e($telefone) ?>" maxlength="20"
required></label>
<label>E-mail<input type="email" name="email" value="<?= e($email) ?>" maxlength="120"
required></label>
<button name="acao" value="salvar">Salvar</button> <a href="tutores.php">Cancelar / Novo</a>
</form>
<h2>Registros cadastrados</h2>
<div class="rolagem"><table><thead><tr><th>ID</th><th>Nome</th><th>CPF (11
dígitos)</th><th>Telefone</th><th>E-mail</th><th>Ações</th></tr></thead><tbody>
<?php while ($r = $listagem->fetch_assoc()): ?><tr>
<td><?= e($r['id']) ?></td>
<td><?= e($r['nome']) ?></td>
<td><?= e(descriptografar($r['cpf_criptografado'])) ?></td>
<td><?= e($r['telefone']) ?></td>
<td><?= e($r['email']) ?></td>
<td><a href="tutores.php?editar=<?= e($r['id']) ?>">Editar</a>
<form method="post" onsubmit="return confirm('Excluir este registro?')">
 <input type="hidden" name="csrf" value="<?= e(token()) ?>">
 <input type="hidden" name="id" value="<?= e($r['id']) ?>">
 <button name="acao" value="excluir">Excluir</button>
</form></td></tr>
<?php endwhile; ?></tbody></table></div>
<?php rodape(); ?>
```

Como o arquivo funciona

Inicialização

Inclui proteção e conexão. Variáveis vazias representam um novo cadastro. $id = 0 significa INSERT; ID positivo representa edição.

Exclusão

No POST com acao=excluir, DELETE usa ID preparado. A FK bloqueia tutor que possui pets, e o catch apresenta uma mensagem ao usuário.

Validações

Verifica campos, limites, 11 dígitos do CPF e formato de e-mail. O código valida no servidor além do required do HTML.

Salvamento

CPF é cifrado antes de gravar. UPDATE recebe nome, CPF cifrado, telefone, e-mail e ID; INSERT recebe os quatro dados. A quantidade e ordem dos parâmetros devem combinar com ?.

Edição

GET editar seleciona pelo ID. fetch_assoc lê o registro. O CPF é decifrado antes de preencher o campo visível. Se o ID não existe, mostra aviso e volta a novo registro.

Busca

GET busca vira %termo%. Três parâmetros iguais procuram nome, e-mail e telefone com LIKE. A pesquisa vazia gera %% e lista todos. O CPF cifrado não é pesquisado com LIKE.

Listagem

while lê cada linha e e() protege a saída. Editar usa GET apenas para carregar o formulário. Excluir utiliza POST e confirmação.

Após gravar

O redirecionamento evita reenviar o formulário quando o aluno atualiza a página. A query ok=1 apresenta sucesso.

Teste

Cadastre tutor, busque pelo nome, edite telefone e exclua um tutor sem pets. Em seguida confirme o bloqueio quando existir pet associado.

25. Crie pets.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva pets.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a pets.php.

```
<?php
require_once 'proteger.php';
require_once 'conexao.php';
$erro = '';
$id = 0;
$nome = '';
$especie = '';
$raca = '';
$data_nascimento = '';
$tutor_id = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    validarToken();
    $id = filter_var($_POST['id'] ?? 0, FILTER_VALIDATE_INT);
    try {
        if ($id === false || $id < 0) { throw new RuntimeException('ID inválido.'); }
        if (($_POST['acao'] ?? '') === 'excluir') {
            $stmt = $conn->prepare('DELETE FROM pets WHERE id = ?');
            $stmt->bind_param('i', $id);
            $stmt->execute();
            header('Location: pets.php?ok=1'); exit;
        }
        $nome = trim($_POST['nome'] ?? '');
        if ($nome === '') { throw new RuntimeException('Preencha Nome.'); }
        if (mb_strlen($nome) > 100) { throw new RuntimeException('Campo Nome muito longo.'); }
        $especie = trim($_POST['especie'] ?? '');
        if ($especie === '') { throw new RuntimeException('Preencha Espécie.'); }
        if (mb_strlen($especie) > 50) { throw new RuntimeException('Campo Espécie muito longo.'); }
        $raca = trim($_POST['raca'] ?? '');
        if ($raca === '') { throw new RuntimeException('Preencha Raça.'); }
        if (mb_strlen($raca) > 80) { throw new RuntimeException('Campo Raça muito longo.'); }
        $data_nascimento = trim($_POST['data_nascimento'] ?? '');
        if ($data_nascimento === '') { throw new RuntimeException('Preencha Nascimento.'); }
        $tutor_id = trim($_POST['tutor_id'] ?? '');
        if ($tutor_id === '') { throw new RuntimeException('Preencha Tutor.'); }
        if (!dataValida($data_nascimento, 'Y-m-d') || $data_nascimento > date('Y-m-d')) {
            throw new RuntimeException('Data de nascimento inválida.');
        }
        $tutor_id = filter_var($tutor_id, FILTER_VALIDATE_INT);
        if (!$tutor_id || $tutor_id < 1) { throw new RuntimeException('Selecione um vínculo válido.');
}
        $check = $conn->prepare('SELECT id FROM tutores WHERE id = ?');
        $check->bind_param('i', $tutor_id);
        $check->execute();
        if (!$check->get_result()->fetch_assoc()) {
            throw new RuntimeException('Registro relacionado não existe.');
        }
        if ($id > 0) {
            $stmt = $conn->prepare('UPDATE pets SET nome = ?, especie = ?, raca = ?, data_nascimento =
?, tutor_id = ? WHERE id = ?');
            $stmt->bind_param('ssssii', $nome, $especie, $raca, $data_nascimento, $tutor_id, $id);
        } else {
            $stmt = $conn->prepare('INSERT INTO pets (nome, especie, raca, data_nascimento, tutor_id)
VALUES (?, ?, ?, ?, ?)');
            $stmt->bind_param('ssssi', $nome, $especie, $raca, $data_nascimento, $tutor_id);
        }
        $stmt->execute();
        header('Location: pets.php?ok=1'); exit;
    } catch (mysqli_sql_exception $ex) {
        error_log($ex->getMessage());
        $erro = 'Operação não concluída. Verifique os vínculos: registros com dependências não podem
ser excluídos.';
    } catch (RuntimeException $ex) {
        $erro = $ex->getMessage();
    }
```

```
}
if (isset($_GET['editar']) && $_SERVER['REQUEST_METHOD'] !== 'POST') {
    $id = intval($_GET['editar']);
    $stmt = $conn->prepare('SELECT * FROM pets WHERE id = ?');
    $stmt->bind_param('i', $id);
    $stmt->execute();
    $registro = $stmt->get_result()->fetch_assoc();
    if (!$registro) { $id = 0; $erro = 'Registro não encontrado.'; }
    else {
        $nome = $registro['nome'];
        $especie = $registro['especie'];
        $raca = $registro['raca'];
        $data_nascimento = $registro['data_nascimento'];
        $tutor_id = $registro['tutor_id'];
    }
}
$opcoes = $conn->query('SELECT id, nome FROM tutores ORDER BY nome');
$listagem = $conn->query('SELECT p.*, t.nome AS tutor FROM pets p INNER JOIN tutores t ON t.id =
p.tutor_id ORDER BY p.nome');
cabecalho('Pets');
?>
<p class="aviso"><?= e($erro) ?></p>
<?php if (isset($_GET['ok'])): ?><p>Operação concluída.</p><?php endif; ?>
<h2><?= $id ? 'Editar registro' : 'Novo registro' ?></h2>
<form method="post">
<input type="hidden" name="csrf" value="<?= e(token()) ?>">
<input type="hidden" name="id" value="<?= e($id) ?>">
<label>Nome<input type="text" name="nome" value="<?= e($nome) ?>" maxlength="100" required></label>
<label>Espécie<input type="text" name="especie" value="<?= e($especie) ?>" maxlength="50"
required></label>
<label>Raça<input type="text" name="raca" value="<?= e($raca) ?>" maxlength="80" required></label>
<label>Nascimento<input type="date" name="data_nascimento" value="<?= e($data_nascimento) ?>"
required></label>
<label>Tutor<select name="tutor_id" required><option value="">Selecione</option>
<?php while ($o = $opcoes->fetch_assoc()): ?>
<option value="<?= e($o['id']) ?>" <?= (int)$tutor_id === (int)$o['id'] ? 'selected' : '' ?>><?=
e($o['nome']) ?></option>
<?php endwhile; ?></select></label>
<button name="acao" value="salvar">Salvar</button> <a href="pets.php">Cancelar / Novo</a>
</form>
<h2>Registros cadastrados</h2>
<div class="rolagem"><table><thead><tr><th>ID</th><th>Nome</th><th>Espécie</th><th>Raça</th><th>Nascime
nto</th><th>Tutor</th><th>Ações</th></tr></thead><tbody>
<?php while ($r = $listagem->fetch_assoc()): ?><tr>
<td><?= e($r['id']) ?></td>
<td><?= e($r['nome']) ?></td>
<td><?= e($r['especie']) ?></td>
<td><?= e($r['raca']) ?></td>
<td><?= e($r['data_nascimento']) ?></td>
<td><?= e($r['tutor']) ?></td>
<td><a href="pets.php?editar=<?= e($r['id']) ?>">Editar</a>
<form method="post" onsubmit="return confirm('Excluir este registro?')">
 <input type="hidden" name="csrf" value="<?= e(token()) ?>">
 <input type="hidden" name="id" value="<?= e($r['id']) ?>">
 <button name="acao" value="excluir">Excluir</button>
</form></td></tr>
<?php endwhile; ?></tbody></table></div>
<?php rodape(); ?>
```

Como o arquivo funciona

Vínculo obrigatório

O formulário possui select de tutor. value envia o ID; o texto mostra o nome. Sem tutor válido, o servidor rejeita o cadastro.

Validações

Nome, espécie, raça e nascimento são obrigatórios. dataValida recusa uma data inexistente e a comparação recusa nascimento futuro.

FK no servidor

filter_var exige inteiro no tutor_id. O SELECT confirma que o tutor ainda existe antes de gravar. A própria FK também garante integridade.

INSERT e UPDATE

Os quatro campos textuais usam s, tutor_id usa i. UPDATE acrescenta id no fim. O nome tutor_id precisa ser igual ao do modelo Workbench.

Listagem com JOIN

p.* traz dados de pets e t.nome AS tutor traz o nome do responsável. p e t são apelidos de tabela, definidos no FROM.

Exclusão

Pet com consultas é bloqueado pela FK de agendamentos. Pet sem consultas pode ser excluído.

Teste

Cadastre um animal para o tutor novo, altere a raça e confirme o nome do tutor na listagem. Tente nascimento futuro e exclusão com consulta.

26. Crie agendamentos.php

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva agendamentos.php.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a agendamentos.php.

```
<?php
require_once 'proteger.php';
require_once 'conexao.php';
$erro = '';
$id = 0;
$pet_id = '';
$data_hora = '';
$motivo = '';
$observacoes = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    validarToken();
    $id = filter_var($_POST['id'] ?? 0, FILTER_VALIDATE_INT);
    try {
        if ($id === false || $id < 0) { throw new RuntimeException('ID inválido.'); }
        if (($_POST['acao'] ?? '') === 'excluir') {
            $stmt = $conn->prepare('DELETE FROM agendamentos WHERE id = ?');
            $stmt->bind_param('i', $id);
            $stmt->execute();
            header('Location: agendamentos.php?ok=1'); exit;
        }
        $pet_id = trim($_POST['pet_id'] ?? '');
        if ($pet_id === '') { throw new RuntimeException('Preencha Pet.'); }
        $data_hora = trim($_POST['data_hora'] ?? '');
        if ($data_hora === '') { throw new RuntimeException('Preencha Data e horário.'); }
        $motivo = trim($_POST['motivo'] ?? '');
        if ($motivo === '') { throw new RuntimeException('Preencha Motivo.'); }
        if (mb_strlen($motivo) > 255) { throw new RuntimeException('Campo Motivo muito longo.'); }
        $observacoes = trim($_POST['observacoes'] ?? '');
        if (mb_strlen($observacoes) > 2000) { throw new RuntimeException('Campo Observações muito
longo.'); }
        if (!dataValida($data_hora, 'Y-m-d\TH:i')) {
            throw new RuntimeException('Data e horário inválidos.');
        }
        $data_hora = str_replace('T', ' ', $data_hora) . ':00';
        $pet_id = filter_var($pet_id, FILTER_VALIDATE_INT);
        if (!$pet_id || $pet_id < 1) { throw new RuntimeException('Selecione um vínculo válido.'); }
        $check = $conn->prepare('SELECT id FROM pets WHERE id = ?');
        $check->bind_param('i', $pet_id);
        $check->execute();
        if (!$check->get_result()->fetch_assoc()) {
            throw new RuntimeException('Registro relacionado não existe.');
        }
        if ($id > 0) {
            $stmt = $conn->prepare('UPDATE agendamentos SET pet_id = ?, data_hora = ?, motivo = ?,
observacoes = ? WHERE id = ?');
            $stmt->bind_param('isssi', $pet_id, $data_hora, $motivo, $observacoes, $id);
        } else {
            $stmt = $conn->prepare('INSERT INTO agendamentos (pet_id, data_hora, motivo, observacoes)
VALUES (?, ?, ?, ?)');
            $stmt->bind_param('isss', $pet_id, $data_hora, $motivo, $observacoes);
        }
        $stmt->execute();
        header('Location: agendamentos.php?ok=1'); exit;
    } catch (mysqli_sql_exception $ex) {
        error_log($ex->getMessage());
        $erro = 'Operação não concluída. Verifique os vínculos: registros com dependências não podem
ser excluídos.';
    } catch (RuntimeException $ex) {
        $erro = $ex->getMessage();
    }
}
if ($data_hora !== '') { $data_hora = substr(str_replace(' ', 'T', $data_hora), 0, 16); }
if (isset($_GET['editar']) && $_SERVER['REQUEST_METHOD'] !== 'POST') {
    $id = intval($_GET['editar']);
```

```
    $stmt = $conn->prepare('SELECT * FROM agendamentos WHERE id = ?');
    $stmt->bind_param('i', $id);
    $stmt->execute();
    $registro = $stmt->get_result()->fetch_assoc();
    if (!$registro) { $id = 0; $erro = 'Registro não encontrado.'; }
    else {
        $pet_id = $registro['pet_id'];
        $data_hora = substr(str_replace(' ', 'T', $registro['data_hora']), 0, 16);
        $motivo = $registro['motivo'];
        $observacoes = $registro['observacoes'];
    }
}
$opcoes = $conn->query('SELECT p.id, p.nome, t.nome AS tutor FROM pets p INNER JOIN tutores t ON t.id =
 p.tutor_id ORDER BY p.nome');
$listagem = $conn->query('SELECT a.*, p.nome AS pet, p.especie, p.raca,
    p.data_nascimento, p.tutor_id, t.nome AS tutor, t.cpf_criptografado,
    t.telefone, t.email FROM agendamentos a
    INNER JOIN pets p ON p.id = a.pet_id
    INNER JOIN tutores t ON t.id = p.tutor_id ORDER BY a.data_hora ASC, a.id ASC');
cabecalho('Agendamentos');
?>
<p class="aviso"><?= e($erro) ?></p>
<?php if (isset($_GET['ok'])): ?><p>Operação concluída.</p><?php endif; ?>
<h2><?= $id ? 'Editar registro' : 'Novo registro' ?></h2>
<form method="post">
<input type="hidden" name="csrf" value="<?= e(token()) ?>">
<input type="hidden" name="id" value="<?= e($id) ?>">
<label>Pet<select name="pet_id" required><option value="">Selecione</option>
<?php while ($o = $opcoes->fetch_assoc()): ?>
<option value="<?= e($o['id']) ?>" <?= (int)$pet_id === (int)$o['id'] ? 'selected' : '' ?>><?=
e($o['nome'] . ' - ' . $o['tutor']) ?></option>
<?php endwhile; ?></select></label>
<label>Data e horário<input type="datetime-local" name="data_hora" value="<?= e($data_hora) ?>"
required></label>
<label>Motivo<input type="text" name="motivo" value="<?= e($motivo) ?>" maxlength="255"
required></label>
<label>Observações<input type="text" name="observacoes" value="<?= e($observacoes) ?>"
maxlength="2000"></label>
<button name="acao" value="salvar">Salvar</button> <a href="agendamentos.php">Cancelar / Novo</a>
</form>
<h2>Registros cadastrados</h2>
<div class="rolagem"><table><thead><tr><th>Agendamento ID</th><th>Pet
ID</th><th>Pet</th><th>Espécie</th><th>Raça</th><th>Nascimento</th><th>Tutor ID</th><th>Tutor</th><th>C
PF</th><th>Telefone</th><th>E-
mail</th><th>Horário</th><th>Motivo</th><th>Observações</th><th>Ações</th></tr></thead><tbody>
<?php while ($r = $listagem->fetch_assoc()): ?><tr>
<td><?= e($r['id']) ?></td>
<td><?= e($r['pet_id']) ?></td>
<td><?= e($r['pet']) ?></td>
<td><?= e($r['especie']) ?></td>
<td><?= e($r['raca']) ?></td>
<td><?= e($r['data_nascimento']) ?></td>
<td><?= e($r['tutor_id']) ?></td>
<td><?= e($r['tutor']) ?></td>
<td><?= e(descriptografar($r['cpf_criptografado'])) ?></td>
<td><?= e($r['telefone']) ?></td>
<td><?= e($r['email']) ?></td>
<td><?= e($r['data_hora']) ?></td>
<td><?= e($r['motivo']) ?></td>
<td><?= e($r['observacoes']) ?></td>
<td><a href="agendamentos.php?editar=<?= e($r['id']) ?>">Editar</a>
<form method="post" onsubmit="return confirm('Excluir este registro?')">
 <input type="hidden" name="csrf" value="<?= e(token()) ?>">
 <input type="hidden" name="id" value="<?= e($r['id']) ?>">
 <button name="acao" value="excluir">Excluir</button>
</form></td></tr>
<?php endwhile; ?></tbody></table></div>
<?php rodape(); ?>
```

Como o arquivo funciona

Seleção do paciente

O select exibe pet e tutor para distinguir animais de nomes iguais, mas envia somente pet_id.

Data e horário

datetime-local envia formato com T, por exemplo 2026-10-10T08:00. A validação confirma o formato; str_replace converte para DATETIME com espaço e segundos.

Gravação

pet_id usa i, data_hora/motivo/observacoes usam s. UPDATE recebe id ao final. Observações podem ficar vazias, mas o valor enviado ao banco é texto vazio, não NULL.

Edição

Ao carregar o registro, converte o DATETIME do banco novamente para o formato do input. A alteração do horário passa pelo mesmo UPDATE.

Listagem completa

INNER JOIN liga a.pet_id a p.id e p.tutor_id a t.id. A consulta seleciona os campos de consulta, animal e tutor; o CPF é decifrado na saída protegida.

Ordenação

ORDER BY a.data_hora ASC coloca as consultas em ordem crescente. a.id é o segundo critério para horários iguais.

Teste

Cadastre dois horários em ordem inversa e veja a ordem final. Edite a data, confira os dados relacionados e exclua a consulta criada para teste.

27. Crie style.css

1. No editor, dentro da pasta sistema, clique em Novo Arquivo e escreva style.css.

2. Copie o conteúdo integral abaixo. Não adicione código de outro arquivo antes ou depois dele.

3. Salve em UTF-8 sem BOM. Confira que a extensão corresponde a style.css.

```
* { box-sizing: border-box; }
body { margin: 0; background: #eef3f7; color: #203040;
 font-family: Arial, sans-serif; }
main { max-width: 1200px; margin: 30px auto; padding: 24px;
 background: white; border-radius: 12px; }
nav { display: flex; flex-wrap: wrap; gap: 18px; padding: 15px 0; }
a { color: #086391; }
form { padding: 12px 0; }
label { display: block; margin: 12px 0; }
input, select { display: block; width: 100%; max-width: 520px;
 padding: 10px; border: 1px solid #aab8c4; border-radius: 5px; }
button { padding: 10px 16px; background: #086391; color: white;
 border: 0; border-radius: 5px; cursor: pointer; }
.aviso { color: #a22020; }
.rolagem { overflow-x: auto; }
table { border-collapse: collapse; width: 100%; }
th, td { padding: 10px; text-align: left; border: 1px solid #ccd6df; }
th { background: #e6f1f8; }
td form { padding: 6px 0; }
@media (max-width: 600px) { main { margin: 0; padding: 15px; } }
```

Como o arquivo funciona

Aplicação

O link criado no cabeçalho carrega esta folha. As regras ajustam fundo, main, formulários, botões e tabela.

Tabela extensa

A classe rolagem fornece overflow-x auto para os dados completos dos agendamentos. A media query adapta espaços no celular.

Teste

Se o visual não carregar, confira nome style.css, localização ao lado de funcoes.php e o link do cabeçalho. Atualize com Ctrl+F5 depois de alterar o CSS.

28. Entenda bind_param para não inverter parâmetros

Tipo	Significado	Exemplo
s	String: texto, data ou hash	nome, email, data_hora
i	Inteiro	id, tutor_id, pet_id
d	Número decimal	Não utilizado nesta aplicação
b	Dados binários enviados separadamente	Não utilizado nesta aplicação

```
$sql = "SELECT * FROM tutores WHERE nome LIKE ? OR email LIKE ?";
$stmt = $conn->prepare($sql);
$termo = "%" . $busca . "%";
$stmt->bind_param("ss", $termo, $termo);
$stmt->execute();
```

O primeiro s corresponde ao primeiro ?, que busca no nome. O segundo s corresponde ao segundo ?, que busca no e-mail. $termo aparece duas vezes porque a mesma busca deve ser aplicada aos dois campos. prepared statement separa a estrutura SQL dos valores.

29. Execute os testes na ordem correta

Teste	Ação do aluno	Resultado esperado
T01	Acessar sistema e digitar senha errada	Aviso de falha; sem acesso privado
T02	Entrar com admin / 123456	Principal com nome Administrador
T03	Cadastrar e buscar tutor	Registro aparece e busca filtra a lista
T04	Editar tutor	Informações atualizadas após recarregar
T05	Cadastrar pet associado	Pet aparece com nome do tutor
T06	Enviar nascimento futuro	Aviso, sem gravar data inválida
T07	Criar consultas fora de ordem	Lista ordenada por data e horário
T08	Editar horário de consulta	Nova data persistida
T09	Tentar excluir tutor/pet com dependências	Operação bloqueada
T10	Excluir consulta, depois pet, depois tutor de teste	Exclusões permitem seguir a ordem
T11	Sair e acessar uma URL privada	Retorna ao login
T12	Expirar a sessão	Solicita nova autenticação
T13	Consultar tabelas pelo Workbench	Hashes e CPFs protegidos; no mínimo três registros
T14	Reabrir ZIP final	Arquivos completos e corretos

Para T12, altere temporariamente TEMPO_SESSAO para 10, faça login, espere mais de 10 segundos sem navegar e abra uma página protegida. Depois restaure 900. Evite deixar o limite de teste na entrega.

Crie registros extras para os testes destrutivos. Não apague os únicos três exemplos de uma tabela antes de gerar o SQL final. Documente cada teste com entrada, resultado observado, situação e data. Não marque como aprovado antes de executar.

30. Corrija os problemas mais comuns

Sintoma	O que conferir
Erro 3780 ou FK incompatível	INT e UN iguais no id pai e na FK filha
Erro 1822	Coluna referenciada possui PK ou índice apropriado
Erro 1452	Tutor/pet pai existe antes de inserir o registro filho
Erro 1068	Uma só PK por tabela; neste exemplo apenas id
Unknown column	Mesmo nome no modelo, no banco e no PHP; tutor_id e pet_id
Table already exists	Você criou por Forward Engineer e tentou executar CREATE novamente
Access denied	Credenciais do config.php e da conexão Workbench
Página não encontrada	Pasta na raiz web, nome exato e Apache iniciado
Headers already sent	UTF-8 sem BOM; nenhuma saída antes de sessão/header
mb_strlen indefinida	Habilitar mbstring e reiniciar servidor web
get_result indisponível	Instalação PHP precisa de mysqlnd
CPF vazio na tela	Chave SEGREDO_CPF e conteúdo cifrado correspondem
Falha no Data Export	Compatibilidade do servidor com os utilitários usados pelo Workbench

31. Termine a documentação e organize a entrega

1. Use documentacao.pdf do pacote como referência e organize os dados no Anexo 1 caso o avaliador forneça o modelo obrigatório.

2. Inclua identificação, problema, objetivo, RF01 a RF09, regras, modelo de dados e os casos de teste realmente executados.

3. Registre versões reais: SELECT VERSION() no Workbench para o banco, php -v no terminal para PHP e winver para Windows. Não confunda a versão do Workbench com a do servidor.

4. Exporte o DER do seu modelo e confira correspondência com o banco final.

5. Exporte saep_db.sql com estrutura e registros. Abra o arquivo e valide as quatro tabelas, as duas FKs e os INSERTs.

6. Copie a pasta de execução sistema para dentro da pasta final com o nome completo do aluno. Não entregue somente o atalho do editor ou um repositório vazio.

7. Dentro de sistema, confira os 12 nomes da lista. Não existe instalar_dados.php nesta versão: os INSERTs já estão protegidos.

8. Na raiz da pasta final, confira documentacao.pdf, DER_clinica.pdf/.png e saep_db.sql. O .mwb pode acompanhar como apoio, se permitido.

9. Compacte a pasta inteira em ZIP e deixe o arquivo na área de trabalho, conforme a prova.

10. Abra o ZIP e verifique que contém a pasta sistema e os arquivos reais, não atalhos. Se possível, teste a restauração em ambiente separado antes de entregar.

Separação do material de aula e da entrega

O guia, 01_estrutura_referencia.sql e 02_populacao.sql são materiais de apoio. A entrega obrigatória usa um único saep_db.sql com estrutura e dados, o DER, a documentação e a pasta sistema. O aluno não precisa incluir o PDF do roteiro de estudo no ZIP da avaliação.

Checklist final

[ ] Banco e PHP usam saep_db.

[ ] Quatro tabelas com ao menos três registros em cada uma.

[ ] Login correto funciona e login errado mostra mensagem.

[ ] Principal mostra usuário e links; Sair encerra a sessão.

[ ] Sessão possui expiração e páginas privadas estão protegidas.

[ ] Tutor possui busca, cadastro, edição, exclusão e CPF cifrado.

[ ] Pet exige tutor; agendamento exige pet.

[ ] Agendamentos estão ordenados e mostram dados completos relacionados.

[ ] DER, SQL e PHP usam os mesmos nomes de campos.

[ ] documentacao.pdf e fontes PHP/CSS estão presentes no ZIP.

32. Referências e verificação deste material

Fonte do desafio: saep_2026.pdf da clínica veterinária, recebido nesta conversa. O material base fornece as regras e o formato de entrega. O novo roteiro adapta a construção do banco para modelagem e SQL no Workbench.

MySQL Workbench Manual - Foreign Keys Tab:
https://dev.mysql.com/doc/workbench/en/wb-table-editor-foreign-keys-tab.html

MySQL Workbench Manual - Forward Engineering to a Live Server:
https://dev.mysql.com/doc/workbench/en/wb-forward-engineering-live-server.html

MySQL Workbench Manual - Forward Engineering Using an SQL Script:
https://dev.mysql.com/doc/workbench/en/wb-forward-engineering-sql-scripts.html

MySQL Workbench Manual - SQL Data Export and Import Wizard:
https://dev.mysql.com/doc/workbench/en/wb-admin-export-import-management.html

Foram conferidos o conteúdo integral dos arquivos, dependências, consistência do SQL e dados cifrados, além da diagramação do PDF. Este ambiente não disponibiliza PHP/MySQL nem o aplicativo Workbench; os testes integrados e a execução dos cliques devem ser realizados no laboratório. Os resultados da documentação ficam a executar.